CS-375 Operating Systems Lab 9: Advanced Segmented, Paged Memory Model Simulator with TLB, Multi-Level Paging, and Dynamic Memory Management

Santiago Soto

October 30, 2026

Contents

1 Introduction **2**

2 Design **2**
2.1 Data Structures . . . 2
2.2 Address Translation . . . 2
2.3 Corrections to the Starter Code . . . 3

3 Test Output **4**
3.1 Compilation and Regression Tests . . . 4
3.2 Manual Translations . . . 4
3.3 Batch Processing . . . 5
3.4 Random Address Test . . . 6
3.5 Dynamic Segments . . . 7

4 Experiments **10**
4.1 LRU vs FIFO . . . 10
4.2 TLB Size . . . 11
4.3 Page Size . . . 11
4.4 Valid/Invalid Ratio . . . 12

5 Short-Answer Questions **12**

6 Long-Answer Questions **14**

7 Discussion **20**
7.1 Challenges . . . 20
7.2 Enhancements Beyond the Requirements . . . 20
7.3 Summary of Results . . . 20

8 References **21**

1 Introduction

This report documents a simulator of a segmented, paged virtual memory system. It models a segment table, a two-level page table, a TLB with LRU replacement, demand paging with LRU or FIFO page replacement, and segments created and removed at runtime. The simulator is in `memory_simulator_advanced.cpp`, and the sample inputs are in `segments.txt` and `addresses.txt`.

To build and run the simulator:

```
g++ -o memory_simulator_advanced memory_simulator_advanced.cpp -std=c++11
./memory_simulator_advanced
```

Every result below uses the option `-seed`, so the same command repeats the same run. Experiments use 1000 addresses unless stated otherwise.

2 Design

2.1 Data Structures

Structure	Role
`Segment`	ID, base address, limit in pages, protection and per-segment counters.
`Page`	Frame number, present bit, protection and `last_access`.
`PageTable`	Second level: `getFrameNumber`, `setFrame` and `replacePage`.
`DirectoryTable`	First level: creates each page table the first time it is used.
`TLB`	Fixed-size cache of `seg:dir:page` to frame, with LRU replacement.
`PhysicalMemory`	Frame allocation, the FIFO queue, the LRU list and the owner of each frame.
`SegmentTable`	Coordinates the above and implements `translateAddress`, metrics, the timeline and the log.

Table 1: Main data structures

A segment's limit is its total number of pages. Each second-level table holds 4 pages, so a 16-page segment has directories 0–3 and a 5-page segment has directories 0–1, the second holding a single page.

2.2 Address Translation

`translateAddress(segNum, dirNum, pageNum, offset, accessType)` works as follows:

1. Draw the base latency (random, 1–10 units).
2. Validate the segment, segment protection, directory, page and offset. A failure raises an exception and is recorded with its reason.
3. Check the page's protection. This runs on every access, including TLB hits.
4. Look up the TLB. A hit updates `last_access` and finishes.
5. On a miss, walk both table levels, adding 10 units per level.

6. If the page is not present, `getFrameNumber` raises `PageFaultException`. `translateAddress` catches it, loads the page with `replacePage`, invalidates the victim's TLB entry and adds 100 units. This mirrors a hardware trap followed by a restart.

7. Update the TLB and metrics, then return $base + frame \times pageSize + offset$.

Bounds are checked before the TLB lookup, as in the starter code, so invalid addresses do not count as TLB misses.

2.3 Corrections to the Starter Code

The starter code did not compile, and several of its defects would have invalidated the experiments. Each correction has a regression test in `tests/test_simulator.cpp`.

Starter code problem	**Correction**
Missing `<queue>`, `<algorithm>` and `<chrono>`; `segments` was private but used outside the class.	Added the includes and accessor methods.
Demand page faults were never counted, so LRU and FIFO could not be compared.	`fault_count` now counts demand faults; invalid addresses and violations have their own counters.
An evicted page stayed present, so two pages could share one frame, and its TLB entry stayed valid.	Each frame records its owner; eviction clears that page and its TLB entry.
Page tables were filled with random frames that were never allocated.	Pages only hold frames obtained from `PhysicalMemory`.
TLB hits skipped the page protection check and the `last_access` update.	Both now happen on every access.
A page fault overwrote the page's protection, so a write fault made a read-only page writable.	A page fault no longer changes protection.
`-pagesize` was ignored (1000 was hard-coded).	The configured page size is used throughout.
Segments were indexed by position, so `add 3 ...` could not be reached or removed.	Segments are stored in a map keyed by ID.
An invalid segment caused an out-of-bounds write; a negative `dirNum` never finished allocating.	All values are validated, with detailed messages such as "Invalid dirNum 5, max 3".
`removeSegment` left stale TLB entries and leaked its tables.	It invalidates the TLB and frees every table; `DirectoryTable` has a destructor.
`TLB::put` could evict an entry or add a duplicate when updating a key.	Updates replace the entry in place.
Invalid addresses were produced by multiplying by 0, which yields valid segment, directory and page 0.	Each invalid address now breaks exactly one rule.
LRU used list order, not the required timestamps; time only advanced on TLB misses.	LRU evicts the smallest `last_access`; time advances on every translation.

Table 2: Defects corrected in the starter code

Three design decisions keep or deliberately change the starter's behavior:

- A segment's limit is its total number of pages, as the lab defines it. In the starter code each directory had `limit` pages and the number of directories had no bound, so `dirNum` could never be invalid.

- The physical address formula is unchanged, including the segment base. As a result, two different frames in different segments can produce the same address number. This does not affect any metric.

- The random 1–10 latency is kept, with fixed costs added for table walks and page faults. Without them the TLB could have no effect on latency. These costs are a model; a real disk access costs far more than 100 times a memory access.

3 Test Output

3.1 Compilation and Regression Tests

The program compiles with no warnings under `-Wall -Wextra`. All 73 checks pass with AddressSanitizer and UndefinedBehaviorSanitizer enabled, and Valgrind reports no leaks or memory errors.

```
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging# make clean && make && make test
rm -rf bin *_results.txt memory_log.txt
cd report && latexmk -c report.tex
Rc files read:
  /etc/LatexMk
Latexmk: This is Latexmk, John Collins, 20 November 2021, version: 4.76.
mkdir -p bin
g++ -std=c++11 -Wall -Wextra -o bin/memory_simulator_advanced src/memory_simulator_advanced.cpp
g++ -std=c++11 -Wall -Wextra -g -fsanitize=address,undefined -o bin/test_simulator tests/test_simulator.cpp
./bin/test_simulator
73/73 checks passed
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 1: Compilation and regression tests

3.2 Manual Translations

All manual runs use `segments.txt`, in which pages 0, 1 and 5 of segment 0 and page 0 of segment 1 start in frames 0–3.

Address (seg dir page offset access)	Result	Reason
`0 0 1 250 0`	1250	Resident page, TLB miss
`0 0 1 300 1`	1300	TLB hit, write allowed
`0 1 3 10 0`	4010	Page fault, loaded into frame 4
`1 0 0 0 1`	Protection Violation	Write to read-only segment
`5 0 0 0 0`	Segmentation Fault	Segment 5 does not exist
`0 7 0 0 0`	Invalid dirNum	Segment 0 has directories 0–3
`0 0 0 1500 0`	Offset Fault	Offset exceeds page size 1000

Table 3: Manual translations

```
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging# cd ~/CS375-Lab9-TLB-Multi-Level-Paging
./bin/memory_simulator_advanced --config src/segments.txt --seed 42 <<'EOF' | tail -n 15
translate 0 0 1 250 0
translate 0 0 1 300 1
translate 0 1 3 10 0
translate 1 0 0 0 1
translate 5 0 0 0 0
translate 0 7 0 0 0
translate 0 0 0 1500 0
quit
EOF
memsim> translate 0 0 1 250 0
Physical Address: 1250
memsim> translate 0 0 1 300 1
Physical Address: 1300
memsim> translate 0 1 3 10 0
Physical Address: 4010
memsim> translate 1 0 0 0 1
Error: Protection Violation: Cannot write to read-only segment 1
memsim> translate 5 0 0 0 0
Error: Segmentation Fault: Invalid segment 5
memsim> translate 0 7 0 0 0
Error: Invalid dirNum 7, max 3
memsim> translate 0 0 0 1500 0
Error: Offset Fault: Offset 1500 exceeds page size 1000
memsim> quit
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 2: Manual translations, valid and invalid

3.3 Batch Processing

`addresses.txt` contains 21 addresses, run with 6 frames and a 4-entry TLB. Of these, 15 succeeded and 6 failed: 1 segmentation fault, 3 invalid addresses and 2 protection violations. There were 8 page faults and 6 evictions.

```
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging# ./bin/memory_simulator_advanced --config src/segments.txt --frames 6
--tlb 4 --seed 42 --batch src/addresses.txt > /dev/null
cat batch_results.txt
Address (0,0,0,100,0): Physical=100
Address (0,0,1,200,1): Physical=1200
Address (0,0,0,150,0): Physical=150
Address (0,0,1,250,0): Physical=1250
Address (0,1,1,999,1): Physical=2999
Address (1,0,0,42,0): Physical=23042
Address (1,0,0,43,1): Protection Violation: Cannot write to read-only segment 1
Address (0,0,2,10,0): Physical=4010
Address (0,0,2,10,1): Protection Violation: Cannot write to read-only page 0:0:2
Address (2,0,0,0,1): Physical=45000
Address (2,0,1,500,0): Physical=40500
Address (2,0,2,700,1): Physical=41700
Address (2,1,0,1,0): Physical=42001
Address (2,1,1,2,1): Physical=43002
Address (0,0,0,11,0): Physical=4011
Address (2,1,1,3,0): Physical=43003
Address (7,0,0,0,0): Segmentation Fault: Invalid segment 7
Address (0,9,0,0,0): Invalid dirNum 9, max 3
Address (0,0,8,0,0): Invalid pageNum 8, max 3
Address (2,3,3,1000,0): Offset Fault: Offset 1000 exceeds page size 1000
Address (2,3,3,999,0): Physical=45999
Fault Rate: 28.5714% (6 of 21 addresses failed)
Page Faults: 8
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 3: Contents of batch_results.txt

3.4 Random Address Test

With 1000 addresses, a valid ratio of 0.7, 6 frames and a 4-entry TLB, 691 translations succeeded and 309 failed. There were 170 page faults and a TLB hit rate of 68.74%. The log gives a detailed reason for each failure.

```
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging# ./bin/memory_simulator_advanced --config src/segments.txt --frames 6
--tlb 4 --seed 42 --stress 1000 --valid 0.7 > /dev/null
head -n 12 random_results.txt; echo "..."; tail -n 8 random_results.txt
Address 0: (5,3,3,87,Read) Segmentation Fault: Invalid segment 5
Address 1: (0,3,1,64,Write) Physical=4064
Address 2: (0,3,3,10,Read) Physical=5010
Address 3: (1,0,0,508,Write) Protection Violation: Cannot write to read-only segment 1
Address 4: (0,3,1,718,Read) Physical=4718
Address 5: (0,3,2,423,Read) Physical=423
Address 6: (0,1,1,861,Write) Physical=2861
Address 7: (0,3,1,968,Read) Physical=4968
Address 8: (0,2,1,720,Read) Physical=1720
Address 9: (0,3,3,75,Read) Physical=5075
Address 10: (0,3,1,57,Write) Physical=4057
Address 11: (0,3,2,696,Read) Physical=696
...
Address 999: (1,1,3,786,Read) Physical=23786
Invalid Address Rate: 30.9% (309 of 1000)
Page Fault Rate: 17% (170 of 1000)
  Invalid dirNum: 61
  Invalid pageNum: 48
  Offset Fault: 74
  Protection Violation: 61
  Segmentation Fault: 65
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 4: Start and end of random_results.txt

3.5 Dynamic Segments

Adding segment 3 and touching two of its pages raised utilization from 4 to 6 of 12 frames, and each of its two second-level tables was allocated on first use. Removing the segment returned utilization to 4 of 12 frames (33.33%) and removed its TLB entries. A later access to segment 3 raised a segmentation fault.

```
memsim> map
Memory Map (LRU, page size 1000):
Segment 0: Base=0, Limit=16, Protection=RW, Faults=0
  Directory 0: [P0 F0 t=4 RW] [P1 F1 t=1 RW] [P2 -- RO] [P3 -- RW]
  Directory 1: [P0 -- RW] [P1 F2 t=2 RW] [P2 -- RW] [P3 -- RW]
  Directory 2: (table not allocated)
  Directory 3: (table not allocated)
Segment 1: Base=20000, Limit=8, Protection=RO, Faults=0
  Directory 0: [P0 F3 t=3 RO] [P1 -- RO] [P2 -- RO] [P3 -- RO]
  Directory 1: (table not allocated)
Segment 2: Base=40000, Limit=16, Protection=RW, Faults=0
  Directory 0: (table not allocated)
  Directory 1: (table not allocated)
  Directory 2: (table not allocated)
  Directory 3: (table not allocated)
Segment 3: Base=90000, Limit=5, Protection=RW, Faults=2
  Directory 0: [P0 F4 t=5 RW] [P1 -- RW] [P2 -- RW] [P3 -- RW]
  Directory 1: [P0 F5 t=6 RW]
Physical Frames: 6 allocated, 6 free  (Fn[seg:dir:page] allocated, Fn[free] free)
F0[0:0:0]    F1[0:0:1]    F2[0:1:1]    F3[1:0:0]    F4[3:0:0]    F5[3:1:0]
F6[free]     F7[free]     F8[free]     F9[free]     F10[free]    F11[free]
memsim> quit
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 5: Memory map after adding segment 3

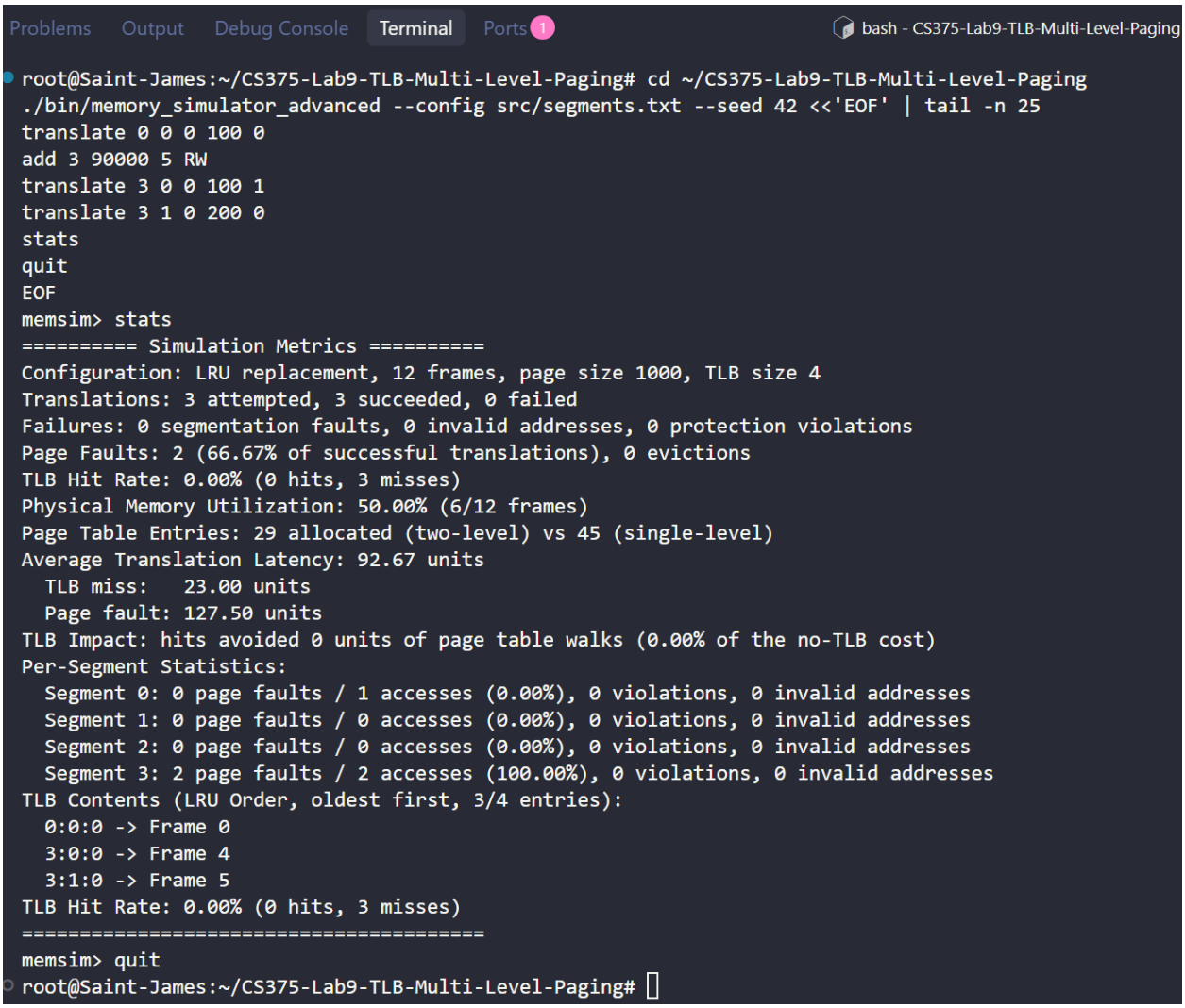


Figure 6: Statistics after adding segment 3

```
memsim> remove 3
Segment 3 removed
Memory Map (LRU, page size 1000):
Segment 0: Base=0, Limit=16, Protection=RW, Faults=0
  Directory 0: [P0 F0 t=4 RW] [P1 F1 t=1 RW] [P2 -- RO] [P3 -- RW]
  Directory 1: [P0 -- RW] [P1 F2 t=2 RW] [P2 -- RW] [P3 -- RW]
  Directory 2: (table not allocated)
  Directory 3: (table not allocated)
Segment 1: Base=20000, Limit=8, Protection=RO, Faults=0
  Directory 0: [P0 F3 t=3 RO] [P1 -- RO] [P2 -- RO] [P3 -- RO]
  Directory 1: (table not allocated)
Segment 2: Base=40000, Limit=16, Protection=RW, Faults=0
  Directory 0: (table not allocated)
  Directory 1: (table not allocated)
  Directory 2: (table not allocated)
  Directory 3: (table not allocated)
Physical Frames: 4 allocated, 8 free  (Fn[seg:dir:page] allocated, Fn[free] free)
F0[0:0:0]    F1[0:0:1]    F2[0:1:1]    F3[1:0:0]    F4[free]     F5[free]
F6[free]     F7[free]     F8[free]     F9[free]     F10[free]    F11[free]
memsim> translate 3 0 0 100 0
Error: Segmentation Fault: Invalid segment 3
memsim> quit
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 7: Memory map after removing segment 3

```
memsim> stats
========== Simulation Metrics ==========
Configuration: LRU replacement, 12 frames, page size 1000, TLB size 4
Translations: 3 attempted, 3 succeeded, 0 failed
Failures: 0 segmentation faults, 0 invalid addresses, 0 protection violations
Page Faults: 2 (66.67% of successful translations), 0 evictions
TLB Hit Rate: 0.00% (0 hits, 3 misses)
Physical Memory Utilization: 33.33% (4/12 frames)
Page Table Entries: 22 allocated (two-level) vs 40 (single-level)
Average Translation Latency: 92.67 units
  TLB miss:   23.00 units
  Page fault: 127.50 units
TLB Impact: hits avoided 0 units of page table walks (0.00% of the no-TLB cost)
Per-Segment Statistics:
  Segment 0: 0 page faults / 1 accesses (0.00%), 0 violations, 0 invalid addresses
  Segment 1: 0 page faults / 0 accesses (0.00%), 0 violations, 0 invalid addresses
  Segment 2: 0 page faults / 0 accesses (0.00%), 0 violations, 0 invalid addresses
TLB Contents (LRU Order, oldest first, 1/4 entries):
  0:0:0 -> Frame 0
TLB Hit Rate: 0.00% (0 hits, 3 misses)
========================================
memsim> quit
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 8: Statistics after removing segment 3

4 Experiments

Each comparison replays one seeded address sequence on a new simulator with memory initially empty, using `segments.txt` (40 pages in total). The *locality* of a workload is the fraction of references sent to a hot region of 20% of one segment, which moves every 100 references. Tables show seed 42; `scripts/seed_sweep.sh` repeats each experiment over seeds 1–10.

4.1 LRU vs FIFO

Workload	Frames	Policy	Page faults	TLB hit %	Avg. latency
Locality 0.8	12	LRU	158	81.50	25.12
	12	FIFO	181	78.90	27.94
Locality 0.8	6	LRU	212	78.80	31.06
	6	FIFO	282	71.80	39.46
Uniform (0.0)	12	LRU	696	20.50	91.12
	12	FIFO	682	20.30	89.76

Table 4: LRU vs FIFO, 8-entry TLB, all addresses valid

Over 10 seeds with 12 frames, LRU averaged 168.7 faults against 201.0 for FIFO at locality 0.8, and had fewer faults on all 10 seeds. On the uniform workload the averages were 678.9 and 678.3, and each policy had fewer faults on 5 seeds.

```
LRU vs FIFO (valid ratio 1, locality 0.8)
1000 addresses, seed 42, 3 segments

Configuration        Frames   PgSize Faults   Fault%   Evict    TLB%   AvgLat   Util%  Failed    PTEs
LRU                      12     1000    158    15.80     146   81.50    25.12  100.00       0      50
FIFO                     12     1000    181    18.10     169   78.90    27.94  100.00       0      50
LRU vs FIFO (valid ratio 1, locality 0.8)
1000 addresses, seed 42, 3 segments

Configuration        Frames   PgSize Faults   Fault%   Evict    TLB%   AvgLat   Util%  Failed    PTEs
LRU                       6     1000    212    21.20     206   78.80    31.06  100.00       0      50
FIFO                      6     1000    282    28.20     276   71.80    39.46  100.00       0      50
LRU vs FIFO (valid ratio 1, locality 0)
1000 addresses, seed 42, 3 segments

Configuration        Frames   PgSize Faults   Fault%   Evict    TLB%   AvgLat   Util%  Failed    PTEs
LRU                      12     1000    696    69.60     684   20.50    91.12  100.00       0      50
FIFO                     12     1000    682    68.20     670   20.30    89.76  100.00       0      50
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 9: LRU vs FIFO on the same workload

4.2 TLB Size

TLB	Seed 42		Mean of 10 seeds	
	Hit %	Avg. latency	Hit %	Avg. latency
Off	0.00	36.72	0.00	37.62
4 entries	70.50	22.62	66.94	24.24
16 entries	86.70	19.38	85.59	20.51

Table 5: TLB size, 20 frames, LRU, locality 0.8, 111 page faults at seed 42 for every size

```
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging# cd ~/CS375-Lab9-TLB-Multi-Level-Paging
./bin/memory_simulator_advanced --config src/segments.txt --seed 42 --valid 1.0 --frames 20 --compare tlb
TLB size comparison (valid ratio 1, locality 0.8)
1000 addresses, seed 42, 3 segments

Configuration        Frames   PgSize Faults   Fault%   Evict    TLB%   AvgLat   Util%  Failed    PTEs
TLB off                  20     1000    111    11.10      91    0.00    36.72  100.00       0      50
TLB 4                    20     1000    111    11.10      91   70.50    22.62  100.00       0      50
TLB 16                   20     1000    111    11.10      91   86.70    19.38  100.00       0      50
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 10: TLB size comparison

4.3 Page Size

Segment sizes and physical memory stay fixed in bytes, at 40 000 bytes of segments and 20 000 bytes of memory, so only the page size changes.

Page size	Frames	Page faults	TLB hit %	Avg. latency	Page table entries
1000	20	111	81.50	20.42	50
4000	5	101	89.90	17.74	13

Table 6: Page size with equal memory, LRU, 8-entry TLB, locality 0.8

```
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging# cd ~/CS375-Lab9-TLB-Multi-Level-Paging
./bin/memory_simulator_advanced --config src/segments.txt --seed 42 --valid 1.0 --frames 20 --tlb 8 --compare pagesize
Page size comparison with equal memory (valid ratio 1, locality 0.8)
1000 addresses, seed 42, 3 segments

Configuration        Frames   PgSize Faults   Fault%   Evict    TLB%   AvgLat   Util%  Failed    PTEs
Page size 1000           20     1000    111    11.10      91   81.50    20.42  100.00       0      50
Page size 4000            5     4000    101    10.10      96   89.90    17.74  100.00       0      13
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 11: Page size comparison

4.4 Valid/Invalid Ratio

Valid ratio	Failed	Page faults	Fault % of successes	TLB hit %	Avg. latency
1.0	0	158	15.80	81.50	25.12
0.9	99	144	15.98	80.80	23.48
0.7	309	125	18.09	78.58	21.08
0.5	506	99	20.04	77.73	17.72
0.1	893	35	32.71	63.55	9.90

Table 7: Valid ratio, 12 frames, LRU, 8-entry TLB, locality 0.8

```
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging# cd ~/CS375-Lab9-TLB-Multi-Level-Paging
./bin/memory_simulator_advanced --config src/segments.txt --seed 42 --frames 12 --tlb 8 --compare ratio
Valid/invalid ratio comparison (locality 0.8)
1000 addresses, seed 42, 3 segments

Configuration        Frames   PgSize Faults   Fault%   Evict    TLB%   AvgLat   Util%  Failed    PTEs
Valid 1                  12     1000    158    15.80     146   81.50    25.12  100.00       0      50
Valid 0.9                12     1000    144    15.98     132   80.80    23.48  100.00      99      50
Valid 0.7                12     1000    125    18.09     113   78.58    21.08  100.00     309      50
Valid 0.5                12     1000     99    20.04      87   77.73    17.72  100.00     506      50
Valid 0.1                12     1000     35    32.71      23   63.55     9.90  100.00     893      50
root@Saint-James:~/CS375-Lab9-TLB-Multi-Level-Paging#
```

Figure 12: Valid/invalid ratio comparison

5 Short-Answer Questions

Question 1

What is the role of the TLB in reducing address translation overhead?

The TLB caches recent page-to-frame translations, so a hit avoids walking the page tables. In the simulator a hit saves 20 latency units, and at a 68.74% hit rate the hits avoided 26.07% of the total translation cost.

Question 2

How does a two-level page table differ from a single-level page table in structure?

A single-level table is one array with an entry for every page, while a two-level table uses a directory that points to smaller page tables. The second-level tables are created only when needed, so unused address ranges use no table memory.

Question 3

Why is LRU often preferred over FIFO for page replacement in modern systems?

LRU uses recent use as a predictor of future use, so it keeps a program's working set in memory, while FIFO can evict a heavily used page just because it was loaded long ago. LRU had fewer page faults on all 10 seeds of the workload with locality.

Question 4

What triggers a segmentation fault vs. a page fault in your simulator?

A segmentation fault is raised when the segment number does not match an existing segment, and it is fatal to that translation. A page fault is raised when a valid page is not present, and it is resolved by loading the page and completing the translation.

Question 5

How does the `present` bit in the page table affect translation?

If the bit is set, the stored frame number is used directly. If it is clear, `getFrameNumber` raises a page fault so the page can be loaded, possibly evicting another page.

Question 6

Why is memory cleanup critical when removing a segment?

Frames that are not freed are lost to every other segment, and stale TLB entries could translate a later segment with the same ID to frames it does not own. Table memory must also be deleted, or the simulator itself leaks memory.

Question 7

What is the purpose of tracking `last_access` in the page table?

It records when each page was last used, so LRU can evict the resident page with the smallest timestamp. It must be updated on TLB hits too, or frequently used pages would look old.

Question 8

How does the protection flag in segments enhance system security?

It lets the system enforce read-only access to regions such as code or shared data, so a buggy or malicious write is stopped before it can change memory. The simulator checks it on every access and logs each violation.

Question 9

Why might a smaller page size increase page faults?

The same data spans more pages, so a program must fault in more pages to touch it. With equal memory, a page size of 1000 caused 111 faults against 101 with a page size of 4000.

Question 10

What is the benefit of logging page fault statistics per segment?

It shows which segment is causing memory pressure, rather than a single overall rate that hides the cause. That lets memory be directed to the segments that need it, and the simulator's suggestions are based on these per-segment rates.

6 Long-Answer Questions

Question 11

Explain how the TLB improves performance in your simulator. Use your code's metrics (e.g., hit rate) to discuss its effectiveness in a scenario with frequent address translations.

The TLB is a small cache in front of the two-level page table. Without it, every translation must read a directory entry and then a page table entry, which the simulator charges at 10 latency units per level. On a hit, `translateAddress` takes the frame directly from the cache, skips the walk and only pays the base latency of 1–10 units. Protection and `last_access` are still handled, so the shortcut does not change behavior, only cost.

The random address test shows the size of the effect. With 1000 addresses, 6 frames and a 4-entry TLB, the hit rate was 68.74% (475 hits and 216 misses). The average latency was 5.75 units for a hit, 24.70 for a miss that needed only a table walk, and 125.75 for a page fault. The simulator reports that the hits avoided 9500 units of table walks, or 26.07% of what the run would have cost without a TLB.

The comparison table isolates the TLB by replaying one workload with its size as the only difference. With 20 frames, every run had 111 page faults, because a TLB does not change which pages are resident. Average latency still fell from 36.72 units with the TLB off to 22.62 with 4 entries and 19.38 with 16 entries. That is a 47% reduction with 16 entries, and it came entirely from avoided walks.

The result depends on locality. A TLB works because programs reuse a small set of pages, so a few entries cover most references. On the uniform workload, with 12 frames and 8 entries, the hit rate fell to about 20%. Frequent translation alone does not make a TLB effective: the references must also repeat. Because page faults dominate latency in this model, the TLB improves the common case, while the page fault rate still sets the worst case.

Question 12

Compare LRU and FIFO page replacement in your simulator. Run tests with both policies on the same workload and analyze differences in page fault rates and performance.

Both policies were tested with `-compare policy`, which replays one seeded address sequence through a new simulator for each policy. Every address was valid, so all differences come from replacement decisions.

On a workload with locality 0.8, 12 frames and an 8-entry TLB, LRU had 158 page faults (15.8%) and FIFO had 181 (18.1%). Average latency was 25.12 units for LRU and 27.94 for FIFO, because each fault adds 100 units. The TLB hit rate also improved, from 78.9% to 81.5%, since eviction removes the victim's TLB entry and fewer evictions leave more useful entries in place. Reducing memory to 6 frames widened the gap: LRU had 212 faults and FIFO had 282, which is 33% more. The less memory there is, the more each wrong eviction costs.

One seed could be a coincidence, so the test was repeated with seeds 1–10. LRU averaged 168.7 faults against 201.0 for FIFO and had fewer faults on all 10 seeds. The reason is that FIFO looks only at load time. A hot page is evicted once it becomes the oldest, however often it is used, and then it immediately faults back in. LRU keeps such a page because every use refreshes its `last_access`.

The advantage disappears without locality. On a uniform workload LRU had 696 faults and FIFO had 682, and over 10 seeds the averages were 678.9 and 678.3, with each policy ahead on 5 seeds. When the past does not predict the future, recency provides no information and the two

policies are equivalent. Near 70% of accesses fault here because 12 frames hold only 30% of the 40 pages.

LRU also costs more. FIFO only updates a queue when a page loads, while LRU updates a timestamp on every access and scans frames to find the minimum. Real systems therefore approximate LRU with schemes such as the clock algorithm.

Question 13

Discuss the challenges of implementing two-level paging. How does it impact memory usage and translation latency compared to single-level paging?

The main design challenge was deciding what a segment's limit means when an address has separate directory and page numbers. In the starter code each directory had `limit` pages and the number of directories was unbounded, so any `dirNum` allocated tables, and a negative value never finished. In this simulator the limit is the total page count, and each table holds 4 pages. That makes both fields checkable, and it allows messages such as "Invalid dirNum 5, max 3". The last table may be partial: a 5-page segment has one page in directory 1, so bounds must be checked against the actual table.

Other challenges concern ownership. Tables are created lazily, so a directory entry may be null and every traversal must handle that. Cleanup must free each table exactly once. Eviction also needs to reach back to a page from its frame, which is why each frame records its owning segment, directory and page.

Memory usage depends on how sparsely a segment is used. A single-level table needs one entry per page, so the three segments in `segments.txt` would always need 40 entries. A two-level design starts with only 10 directory entries. After the dynamic test it held 22 entries, against 40 for a single level. It is not always smaller: in the 1000-address stress test every table was eventually allocated, giving 50 entries against 40, because the directories are extra. Two levels help sparse address spaces, while a densely used one pays a small overhead. With real 32-bit or 64-bit address spaces, which are overwhelmingly sparse, the saving is very large.

The cost is latency. A TLB miss needs two memory references instead of one, so the walk costs 20 units here rather than 10. With the TLB, this matters only on misses. At the 18.5% miss rate measured with an 8-entry TLB, the extra level adds about 1.85 units per access, against an average of 25.12. Without a TLB it would add a full 10 units to every access. This is why multi-level paging and TLBs are used together.

Question 14

Describe how your simulator handles dynamic segment creation and deletion. What measures prevent memory leaks or invalid accesses?

Segments are created with `add <id> <base> <limit> <RO|RW>`. `addSegment` rejects a duplicate ID, a negative ID or base, and a limit outside 1–65536, then stores a `Segment` in a map keyed by ID and creates a `DirectoryTable`. The map matters: the starter code indexed a vector by position, so `add 3` on a two-segment table stored the segment at index 2 and could never remove it. No frames or second-level tables are allocated at creation. They appear on first use, so adding a segment costs only its directory.

Deletion is handled by `removeSegment`, in a specific order. It first walks every allocated table and returns each present page's frame through `freeFrame`, which also removes the frame from the FIFO queue or LRU list and clears its owner. If a frame stayed in those structures, a later

replacement could choose a frame belonging to nobody. Next it calls `invalidateSegment`, which removes all of that segment's TLB entries. Finally it deletes the directory and erases the segment.

Several measures prevent leaks. `DirectoryTable` has a destructor that frees its page tables, a gap in the starter code, and its copy operations are disabled so that tables cannot be freed twice. Valgrind reports that all heap blocks were freed after a session that adds a segment, runs a stress test and removes it.

Several measures prevent invalid accesses. Each translation looks the segment up in the map, so a removed segment raises a segmentation fault instead of reading a blank entry. In the test, `translate 3 0 0 100 0` after removal did exactly this. TLB invalidation stops a stale hit, which would otherwise be the most dangerous failure: a new segment reusing the ID could reach another segment's frames with no error at all. A regression test removes a segment, recreates it with the same ID, and confirms that its first access faults. The memory map also confirms cleanup: after removal, utilization returned from 6 to 4 of 12 frames.

Question 15

Analyze the impact of TLB size on performance. Test with sizes 4 and 16, and discuss trade-offs between hit rate and memory overhead.

TLB sizes were tested with `-compare tlb` on one workload of 1000 valid addresses with locality 0.8, 20 frames and LRU replacement. A disabled TLB was included as a baseline. Over seeds 1–10, the mean hit rate rose from 66.94% with 4 entries to 85.59% with 16, and mean latency fell from 37.62 units with no TLB to 24.24 and then 20.51. Seed 42 behaved the same way: 70.50% and 86.70% hit rates, with latencies of 36.72, 22.62 and 19.38. The page fault count was 111 at seed 42 for all sizes, since the TLB affects only how quickly resident pages are found.

The results show diminishing returns. The first 4 entries saved 13.38 units per access on average. The next 12 entries saved 3.73 units, so each added entry is worth about a tenth of what each of the first four was worth. This follows from the workload: the hot region covers only about 20% of a segment, or roughly 3 to 4 pages, so four entries capture much of it. The remaining entries mostly help with the 20% of references outside that region and with moves of the hot region.

The size that helps also has a natural ceiling. A TLB entry is useful only for a resident page, so with 20 frames a TLB of more than 20 entries could not raise the hit rate here, and a smaller physical memory lowers that limit.

The memory overhead in the simulator is trivial, but in hardware it is the real constraint. A TLB is fully or highly associative, so a lookup compares many entries at once. More entries take more chip area and power and can lengthen the lookup, which runs on every memory reference. A simulator with a fixed hit cost cannot show that penalty. On these numbers, 16 entries is justified because it adds nearly 19 percentage points of hit rate. If each extra entry were expensive, though, 4 entries would already deliver most of the benefit. Real designs resolve this with a small, fast first-level TLB backed by a larger, slower second level.

Question 16

Explain how protection enforcement in your simulator mimics real OS behavior. Provide an example of a protection violation and its handling.

Real systems enforce protection in hardware on every access. The memory management unit compares the access type with permission bits in the page table entry, which the TLB also caches. A denied access traps to the kernel, and the instruction does not complete. On Linux the process normally receives `SIGSEGV`. Protection is therefore not an optional check that fast paths may skip.

The simulator follows this model at two levels. Each segment has a protection flag and each page has its own, so a page may be stricter than its segment but can never be more permissive. For example, page 2 of segment 0 in `segments.txt` is read-only inside a read-write segment. A `ProtectionViolation` exception plays the role of the trap: it stops the translation, so no address is returned, no page is loaded and no frame changes. The check runs before the page fault handler, so a forbidden write cannot bring a page into memory.

Two starter defects would have broken this. A TLB hit returned early, so a write could pass after a read cached the translation. A real TLB holds permission bits for this reason, and here the page's protection is rechecked on each access. A page fault also copied the access type into the page's protection, so a write fault turned a read-only page into a writable one.

As an example, `translate 1 0 0 0 1` is a write to segment 1, which is read-only. The simulator performs these steps:

1. It finds the segment and sees a write to a read-only region.
2. It raises "Protection Violation: Cannot write to read-only segment 1".
3. The handler in `translateAddress` increments the segment's violation counter and the global counter.
4. The event is added to the timeline and `memory_log.txt`, with its time, full logical address and latency.
5. The exception reaches the command loop, which prints the error and continues.

The last step is where the simulator differs from a real OS, which would normally terminate the process. The simulator continues so a batch can report every result, and each violation is logged with its context, much as a kernel records a fault.

Question 17

Evaluate the effectiveness of your stress testing with random addresses. How does varying the valid/invalid ratio reveal system strengths and weaknesses?

Each stress address is either fully valid or breaks exactly one rule, chosen from five: segment, directory, page, offset, or a write to a read-only segment. This replaced the starter's generator, which multiplied values by a Boolean. That produced 0, a valid value, when an address was meant to be invalid, and it compared `mt19937` output with `RAND_MAX`, which has a different range. The test requires a ratio of 1.0 to give no failures and 0.0 to give all failures, and both pass. At seed 42, the failures were 99, 309, 506 and 893 for ratios of 0.9, 0.7, 0.5 and 0.1, close to the expected 100, 300, 500 and 900.

Varying the ratio showed clear strengths. Every invalid address was rejected with a specific reason, and none corrupted state. Even at a ratio of 0.1, with 893 failures, the run finished normally. Sanitizers and a test that checks that no two pages share a frame also pass after stress runs, which is stronger evidence than the output looking plausible.

It also revealed how metrics behave under failure. As the ratio fell from 1.0 to 0.1, average latency dropped from 25.12 to 9.90 units. That looks like an improvement, but failed translations are simply cheap, because they stop before any walk or fault. A rate computed over all attempts would therefore mislead, so the simulator reports page faults relative to successful translations. That rate rose from 15.80% to 32.71%, and the TLB hit rate fell from 81.50% to 63.55%. Fewer

valid references mean that a larger share of them are first-time accesses, and the hot region gets too few references to build up reuse before it moves.

The testing also has weaknesses. The generator never writes to a read-only page inside a read-write segment, so page-level violations occur only in manual and batch runs. Faults cost a fixed amount, and synthetic locality only approximates real programs. Passing also shows only that the checks I wrote held, not that the model is complete: for example, the simulator has no concurrency, so race conditions cannot appear in it.

Question 18

Discuss how page fault frequency tracking can inform memory management decisions. Propose an algorithm to dynamically adjust segment limits based on your simulator's data.

Page fault frequency (PFF) is a direct and inexpensive signal of whether a segment has enough memory. A high rate means that its working set does not fit, so pages are evicted and quickly faulted back in. A very low rate suggests that it holds more memory than it needs. The simulator tracks faults and accesses per segment, and it suggests a larger limit when a segment's rate passes 20% over at least 10 accesses. In the stress test, segments 0 and 2 reached 21.93% and 23.02% and received suggestions, while segment 1 stayed at 12.47%.

The data also shows a limitation in the lab's rule. A segment's limit is the size of its address range, not the memory it holds, so raising a limit does not reduce demand faults. What reduces them is more frames. Invalid-page errors are the signal that a limit is too small. A sound algorithm should separate these cases, so the proposal below adds a per-segment frame quota and evicts within a segment that is at its quota.

```
Every W = 100 translations, for each segment s:
  fault_rate   = faults in window / accesses in window
  bounds_rate  = bounds errors in window / accesses in window

  if bounds_rate > 5%:
      s.limit = min(ceil(s.limit * 1.5), MAX_PAGES)
  if fault_rate > 20%:
      if a free frame exists: s.quota += 1
      else: take 1 frame from the segment with the
            lowest fault_rate, if that rate is below 5%
  else if fault_rate < 5% and s.quota > MIN_QUOTA:
      s.quota -= 1
  if total demand exceeds frames and no donor exists:
      suspend the segment with the highest fault_rate
```

The gap between 5% and 20% is deliberate. With a single threshold, a segment near that value would gain and lose frames at every window. Using a window means that old behavior ages out, and the 10-access minimum already in the simulator avoids acting on tiny samples, such as a first access that counts as a 100% fault rate. Cold-start faults should be discounted for the same reason. The final step handles thrashing: when total demand exceeds memory, taking frames from one segment only moves the problem, so a segment is suspended instead.

Question 19

Propose a real-world application (e.g., IoT device, cloud VM) for your simulator. How would you extend it to meet specific requirements (e.g., real-time deadlines, multi-core support)?

A strong application is capacity planning for a cloud host running many virtual machines. An operator must decide how much memory and TLB capacity each VM needs before committing hardware. Instead of testing on production systems, memory traces from each VM could be replayed through the simulator under different configurations. The existing `-batch` input and `-compare` tables already support this workflow, and page fault and hit rates per segment map naturally to per-VM metrics if each VM is treated as a segment. Three extensions would be needed.

The first is multi-core support. The simulator has one TLB, whereas each core has its own. I would give each core a `TLB` object and run workloads in threads, with locks around shared frame structures. The important change is TLB shootdown: when a page is evicted, every core that may cache it must be told to invalidate that entry, which interrupts the other cores. Modeling that cost would show how eviction scales with core count. Address space identifiers would also be needed, so that a context switch does not require flushing the whole TLB. The current keys already contain a segment number, which is a similar idea.

The second is realistic fault costing. Writing back a modified page costs much more than dropping a clean one, so `Page` needs a dirty bit, set on writes, and the replacement policy should be able to prefer clean victims. Fault latency should also model storage more closely, as a fixed 100 units is far too low for a disk.

The third is real-time support for tenants with latency targets. Average latency is not enough, because one page fault can miss a deadline even when the average looks fine. The simulator would need to report worst-case and 99th-percentile latency. It would also need page locking, with pages excluded from eviction, similar to `mlock`. A deadline could be attached to each request, with misses counted as a metric. Finally, I would replace the LRU scan, which is linear in the number of frames, with a clock algorithm, so that it can handle the large memory sizes that such hosts have.

Question 20

Analyze the trade-offs of page size in your simulator. Test with sizes 1000 and 4000, and discuss impacts on page faults, memory utilization, and translation overhead.

A fair test required care. If only `-pagesize` changes, page and frame counts stay the same, so the fault count cannot change and memory simply becomes four times larger. `-compare pagesize` instead keeps bytes constant: 40 000 bytes of segments and 20 000 bytes of memory. That gives 20 frames at a page size of 1000 and 5 at 4000. The workload is generated as byte positions, so both runs reference the same bytes.

Page faults fell slightly with larger pages. At seed 42 there were 111 faults at 1000 and 101 at 4000. Over seeds 1–10 the means were 121.2 and 105.4, a 13% reduction. A larger page brings in neighboring data, which is prefetching: one fault at 4000 covers what could take four faults at 1000. On the uniform workload the means were 478.8 and 470.8, a gain of under 2%. With no spatial locality, neighboring bytes are rarely needed. There is also a risk that the test understates: with only 5 frames, the simulator can hold just 5 distinct regions, so a workload with many scattered hot spots could fault more with large pages.

Translation overhead improved the most. Each TLB entry now covers four times as much memory, so the reach of 8 entries grows from 8000 to 32 000 bytes. The mean hit rate rose from 80.04% to 89.46%, and on the uniform workload from 21.61% to 52.92%. Mean latency fell from 21.62 to 18.15 units. Page table size also dropped from 50 entries to 13, as fewer pages need describing.

Memory utilization is where the cost appears, and here the metric hides it. Both runs report 100% of frames in use, but that counts frames, not useful bytes. A page is loaded whole even if one byte is needed, so larger pages increase internal fragmentation, and the wasted space is greater

with 4000-byte pages. The simulator does not measure this; it would need tracking of which bytes in a page are used.

Overall, 4000 is better on every metric measured here, but those metrics do not include fragmentation. Real systems balance these factors, commonly using 4 KB pages with larger huge pages for workloads that need more TLB reach.

7 Discussion

7.1 Challenges

- **Consistency after eviction.** A page's state exists in the page table, the frame table and the TLB. The starter code updated only some of these, which let two pages share a frame without any visible error. Recording each frame's owner solved this, and a test now checks for shared frames after stress runs.
- **Making experiments comparable.** The original workload used different random numbers in each run, and the default segments fit in memory, so no replacement ever happened. Seeding, a fixed draw order and a 40-page configuration made comparisons fair.
- **Testing page size fairly.** Changing the page size alone changes the amount of memory. Generating addresses as byte positions and holding memory constant isolated the real effect.
- **Using one random seed.** The first LRU and FIFO comparison on uniform references suggested that FIFO was better. Repeating over 10 seeds showed that the policies were equal, and that the gap was noise.

7.2 Enhancements Beyond the Requirements

- `-compare` prints side-by-side tables from a single seeded workload.
- `-seed` makes every run repeatable.
- A TLB size of 0 turns the TLB off, which gives a measured baseline.
- Second-level tables are allocated on first use, and the simulator reports table memory for two-level and single-level designs.
- Stress workloads have adjustable locality with `-locality`.
- Exceptions form a hierarchy, and a page fault is handled as a trap and retry.
- A regression suite runs under AddressSanitizer and UndefinedBehaviorSanitizer.

7.3 Summary of Results

LRU beat FIFO only when the workload had locality. A TLB cut average latency by up to 47% at seed 42 without affecting page faults, with most of the gain coming from the first few entries. Larger pages improved TLB reach and reduced faults slightly, at a cost in internal fragmentation that the simulator does not measure. Two-level paging saves table memory only in sparse address spaces, and its extra latency is acceptable only because of the TLB. Latency values come from a simple cost model, so the comparisons between configurations are more meaningful than the absolute numbers.

8 References

1. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, chapters on TLBs, advanced page tables and swapping policies. `https://pages.cs.wisc.edu/~remzi/OSTEP/`

2. P. J. Denning, "The Working Set Model for Program Behavior," *Communications of the ACM*, vol. 11, no. 5, 1968.

3. W. W. Chu and H. Opderbeck, "The Page Fault Frequency Replacement Algorithm," *Proceedings of the AFIPS Fall Joint Computer Conference*, 1972.

4. CS375 Lab 9 handout and starter code.